

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

*Московский институт электроники и математики им. А.Н.Тихонова
Департамент электронной инженерии*

«Контейнеризация»

Практическая работа №1
по курсу «Архитектура компьютера и Системное программирование»
студента образовательной программы бакалавриата
«Информационная Безопасность»

Выполнил: Студент группы

БИБ233 Гутников Д.А.

Преподаватель:

Балескин В.А.

1. Введение

В данной лабораторной работе изучается работа контейнеров в операционной системе Linux: контейнеры будут создаваться вручную на основе ручных вызовов функций ядра Linux, полуавтоматически с помощью утилиты `runc` и с помощью платформы для запуска контейнеров Docker.

2. Основная часть

2.1 Создание контейнера из примитивов ядра

Прежде чем создать и запустить контейнер создадим файловую систему для изолированной среды. Для этого хорошо подойдёт небольшая файловая система Alpine Linux, размер которой не превышает 10мб (рисунок 1).

```
gutnikov@sr-gutnikov:~$ mkdir -p /tmp/container/rootfs && cd /tmp/container
gutnikov@sr-gutnikov:/tmp/container$ wget https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpine-minirootfs-3.20.3-x86_64.tar.gz
--2026-04-08 12:43:51-- https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpine-minirootfs-3.20.3-x86_64.tar.gz
Распознаётся dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 199.232.174.132, 2a04:de42:6b::644
Подключение к dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org) [199.232.174.132]:443... соединение установлено.
HTTP-запрос отправлен. Ожидание ответа... 200 OK
Длина: 3490290 (3,3М) [application/octet-stream]
Сохранение в: «alpine-minirootfs-3.20.3-x86_64.tar.gz»

alpine-minirootfs-3.20.3-x86_64.tar.gz 100%[=====] 3,33M 7,36MB/s за 0,5s
2026-04-08 12:43:52 (7,36 MB/s) - «alpine-minirootfs-3.20.3-x86_64.tar.gz» сохранён [3490290/3490290]

gutnikov@sr-gutnikov:/tmp/container$ tar -xzf alpine-minirootfs-3.20.3-x86_64.tar.gz -C rootfs
gutnikov@sr-gutnikov:/tmp/container$ ls rootfs/
bin dev etc home lib media mnt opt proc root run sbin srv sys tmp usr var
gutnikov@sr-gutnikov:/tmp/container$
```

Рисунок 1 – Установка базовой файловой системы.

Для изоляции процессов можно воспользоваться одной из двух команд: unshare и clone. Они отличаются тем, что первая команда создаёт и перемещает новый процесс в уже созданную изолированную область, а unshare отделяет уже созданный процесс в отдельную область.

Воспользуемся командой unshare и перенесём процесс в изолированную область, создав следующие пространства имён:

- PID – пространство имён для процессов;
- UTS – пространство имён имени хоста системы для;
- MNT – пространство имён файловой системы;
- NET – пространство имён сетевых ресурсов;
- IPC – пространство имён межпроцессного взаимодействия.

Если не указывать параметры данных пространств имён, то ядро создаст новые пространства, к которым не получится обращаться напрямую. Изолируем процесс bash (рисунок 2). Перемонтируем файловую систему процессов, чтобы обращаться к процессам нового процесса.

```

gutnikov@sr-gutnikov:/tmp/container$ sudo unshare --ipc --mount --net --pid --uts --fork bash
root@sr-gutnikov:/tmp/container# mount -t proc proc /proc
root@sr-gutnikov:/tmp/container# hostname container
root@sr-gutnikov:/tmp/container# hostname
container
root@sr-gutnikov:/tmp/container# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   6940  3592 pts/0    S   12:58   0:00 bash
root         5  0.0  0.0  10920  3620 pts/0    R+  12:59   0:00 ps aux
root@sr-gutnikov:/tmp/container#

```

Рисунок 2 – Изолирование процесса bash.

По рисунку видна изоляция процесса: имя хоста контейнера изменилось относительно начального хоста (UTS namespace), при вызове «ps aux» видны только процессы текущего пространства имён. Важно отметить, что процесс «ps aux» был создан процессом bash, поэтому он унаследовал пространства имён родителя и оказался в списке процессов.

Примонтируем скачанную файловую систему в данном контейнере, чтобы можно было изолироваться от файловой системы хоста (рисунок 3). Параметр --bind в команде монтирования нужен, чтобы создать точку монтирования rootfs/. Сделав это, можно выполнить команду pivot_root с переносом корня. Путь oldroot/ использовался для команды смены корня и больше не требовался. При монтировании файловой системы процессов используем /bin/busybox. Посмотрим процессы и убедимся в правильной настройке.

```

root@sr-gutnikov:/tmp/container# mount --bind rootfs/ rootfs/
root@sr-gutnikov:/tmp/container# cd rootfs/ && mkdir oldroot
root@sr-gutnikov:/tmp/container/rootfs# pivot_root . oldroot
root@sr-gutnikov:/tmp/container/rootfs# umount -l oldroot/
root@sr-gutnikov:/tmp/container/rootfs# rmdir oldroot/
root@sr-gutnikov:/tmp/container/rootfs# mount -t proc proc /proc
bash: /usr/bin/mount: No such file or directory
root@sr-gutnikov:/tmp/container/rootfs# /bin/busybox mount -t proc proc /proc
root@sr-gutnikov:/tmp/container/rootfs# ps aux
bash: /usr/bin/ps: No such file or directory
root@sr-gutnikov:/tmp/container/rootfs# /bin/busybox ps aux
PID    USER      TIME  COMMAND
    1   root         0:00 bash
   15   root         0:00 /bin/busybox ps aux
root@sr-gutnikov:/tmp/container/rootfs#

```

Рисунок 3 – Монтирование файловой системы.

Посмотрим содержимое каталога дескрипторов процесса 1 изолированного контейнера (рисунок 4).

```

root@sr-gutnikov:/tmp/container/rootfs# ls
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@sr-gutnikov:/tmp/container/rootfs# ls proc/1/ns
cgroup  ipc  mnt  net  pid  pid_for_children  time  time_for_children  user  uts
root@sr-gutnikov:/tmp/container/rootfs#

```

Рисунок 4 – Содержимое процесса 1.

Здесь есть информация о пространствах имён (ipc, mnt, net, pid, uts), а также информация об ограничении ресурсов – cgroup. Посмотрим информацию о процессе из глобальной области и сравним её с информацией из изолированной среды. Для этого узнаем глобальный идентификатор процесса (рисунок 5) и сравним пространства имён (рисунки 6 и 7).

Рисунок 5 – Нахождение процесса.

```

gutnikov@sr-gutnikov:/tmp/container$ sudo ls /proc/2568/ns
cgroup ipc mnt net pid pid_for_children time time_for_children user uts
gutnikov@sr-gutnikov:/tmp/container$ sudo ls /proc/2568/ns -l
итого 0
lrwxrwxrwx 1 root root 0 apr  8 13:08 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 ipc -> 'ipc:[4026532213]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 mnt -> 'mnt:[4026532211]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 net -> 'net:[4026532216]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 pid -> 'pid:[4026532215]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 pid_for_children -> 'pid:[4026532215]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 time -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 apr  8 13:08 uts -> 'uts:[4026532212]'
gutnikov@sr-gutnikov:/tmp/container$

```

Рисунок 6 – Пространства из глобальной области.

```

root@sr-gutnikov:/tmp/container/rootfs# ls
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@sr-gutnikov:/tmp/container/rootfs# ls proc/1/ns
cgroup      mnt         pid         time        user
ipc         net         pid_for_children  time_for_children  uts
root@sr-gutnikov:/tmp/container/rootfs# ls /proc/1/ns -l
total 0
lrwxrwxrwx 1 root root      0 Apr  8 10:06 cgroup -> cgroup:[4026531835]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 ipc -> ipc:[4026532213]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 mnt -> mnt:[4026532211]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 net -> net:[4026532216]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 pid -> pid:[4026532215]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 pid_for_children -> pid:[4026532215]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 time -> time:[4026531834]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 time_for_children -> time:[4026531834]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 user -> user:[4026531837]
lrwxrwxrwx 1 root root      0 Apr  8 10:06 uts -> uts:[4026532212]
root@sr-gutnikov:/tmp/container/rootfs#

```

Рисунок 7 – Пространства имён из изолированной области.

Видно, что числа данных пространств совпадают, что подтверждает тот факт, что мы смотрим на один и тот же процесс с одними и теми же пространствами имён. Для понимания обратимся к другому процессу из глобальной области и рассмотрим его пространства имён и сравним их с пространствами изолированного процесса (рисунок 8).

```

gutnikov@sr-gutnikov:/tmp/container$ sudo ls -l /proc/1/ns
total 0
lrwxrwxrwx 1 root root 0 anp  8 13:39 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 mnt -> 'mnt:[4026531841]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 net -> 'net:[4026531840]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 time -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 anp  8 13:39 uts -> 'uts:[4026531838]'
gutnikov@sr-gutnikov:/tmp/container$ sudo ls -l /proc/2286/ns
total 0
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 mnt -> 'mnt:[4026531841]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 net -> 'net:[4026531840]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 time -> 'time:[4026531834]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 user -> 'user:[4026531837]'
lrwxrwxrwx 1 gutnikov gutnikov 0 anp  8 13:52 uts -> 'uts:[4026531838]'
gutnikov@sr-gutnikov:/tmp/container$ sudo ls -l /proc/2568/ns
total 0
lrwxrwxrwx 1 root root 0 anp  8 13:08 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 ipc -> 'ipc:[4026532213]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 mnt -> 'mnt:[4026532211]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 net -> 'net:[4026532216]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 pid -> 'pid:[4026532215]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 pid_for_children -> 'pid:[4026532215]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 time -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 time_for_children -> 'time:[4026531834]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 anp  8 13:08 uts -> 'uts:[4026532212]'
gutnikov@sr-gutnikov:/tmp/container$

```

Рисунок 8 – Глобальное сравнение процессов.

Как можно увидеть, процессы из глобальной области (выделено синим) имеют одинаковые пространства имён (одинаковые номера), однако процесс из изолированной области (выделен красным цветом) содержит другие числа, причём числа отличаются только в тех местах, которые мы указывали при вызове команды `unshare`.

Теперь перейдём к ограничениям ресурсов `cgroup`. Установим ограничения и проверим их срабатывание. Для начала создадим новую контрольную группу `hse-container` по пути `/sys/fs/cgroup` (рисунок 9) и добавим в неё информацию об ограничениях (рисунок 10).

```
gutnikov@sr-gutnikov:/sys/fs/cgroup$ ls
cgroup.controllers      cgroup.stat             cpuset.cpus.effective  io.cost.model          io.stat                misc.capacity
cgroup.max.depth        cgroup.subtree_control  cpuset.mems.effective  io.cost.qos            memory.numa_stat       system.slice
cgroup.max.descendants    cgroup.threads          cpu.stat               io.pressure            memory.pressure        user.slice
cgroup.procs            cpu.pressure             init.scope             io.prio.class          memory.stat
gutnikov@sr-gutnikov:/sys/fs/cgroup$ sudo mkdir hse-container
[sudo] пароль для gutnikov:
gutnikov@sr-gutnikov:/sys/fs/cgroup$
```

Рисунок 9 – Создание контрольной группы.

```
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$ echo $((50*1024*1024)) | sudo tee memory.max
52428800
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$ echo "10000 100000" | sudo tee cpu.max
10000 100000
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$ echo 20 | sudo tee pids.max
20
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$
```

Рисунок 10 – Заполнение контрольной группы ограничениями.

Теперь процессы, помещённые в данную контрольную группу, будут ограничены:

- Использование памяти не более 50 мб;
- Использование не более 10 % мощности процессора;
- Максимальное число процессов в контрольной группе равно 20.

Чтобы поместить процесс в контрольную группу, поместим его PID в файл `cgroup.procs` (рисунок 11).

```
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$ cat cgroup.procs
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$ echo 2568 | sudo tee cgroup.procs
2568
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$ cat cgroup.procs
2568
gutnikov@sr-gutnikov:/sys/fs/cgroup/hse-container$
```

Рисунок 11 – Помещение процесса в контрольную группу.

Убедимся, что процесс действительно оказался в контрольной группе hse-container (рисунок 12).

```
gutnikov@sr-gutnikov:/proc/2568/ns$ sudo cat /proc/2568/cgroup
0::/hse-container
gutnikov@sr-gutnikov:/proc/2568/ns$
```

Рисунок 12 – Демонстрация помещения процесса в контрольную группу.

Проверим, что правила применились к нашему контейнеру. Попробуем запустить процесс в контексте командной строки (рисунок 13) и посмотрим его нагрузку (рисунок 14).

```
root@sr-gutnikov:/proc/1/ns# yes > /dev/null
```

Рисунок 13 – Запуск процесса.

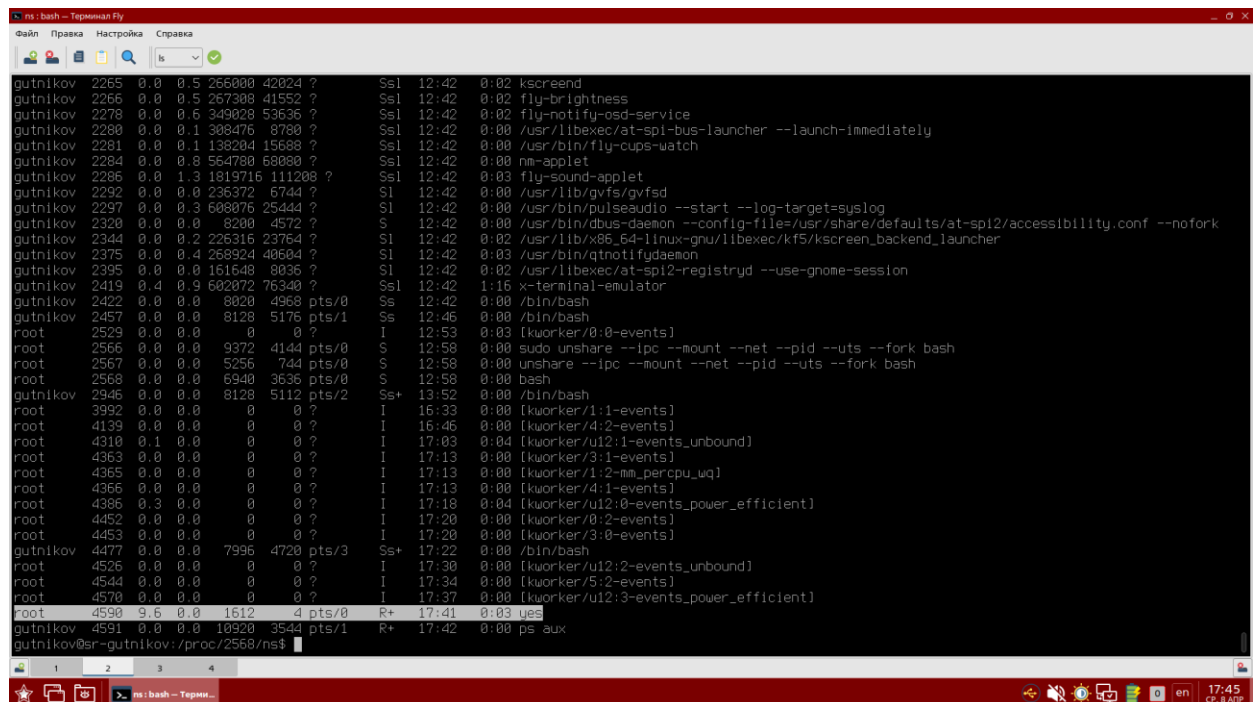
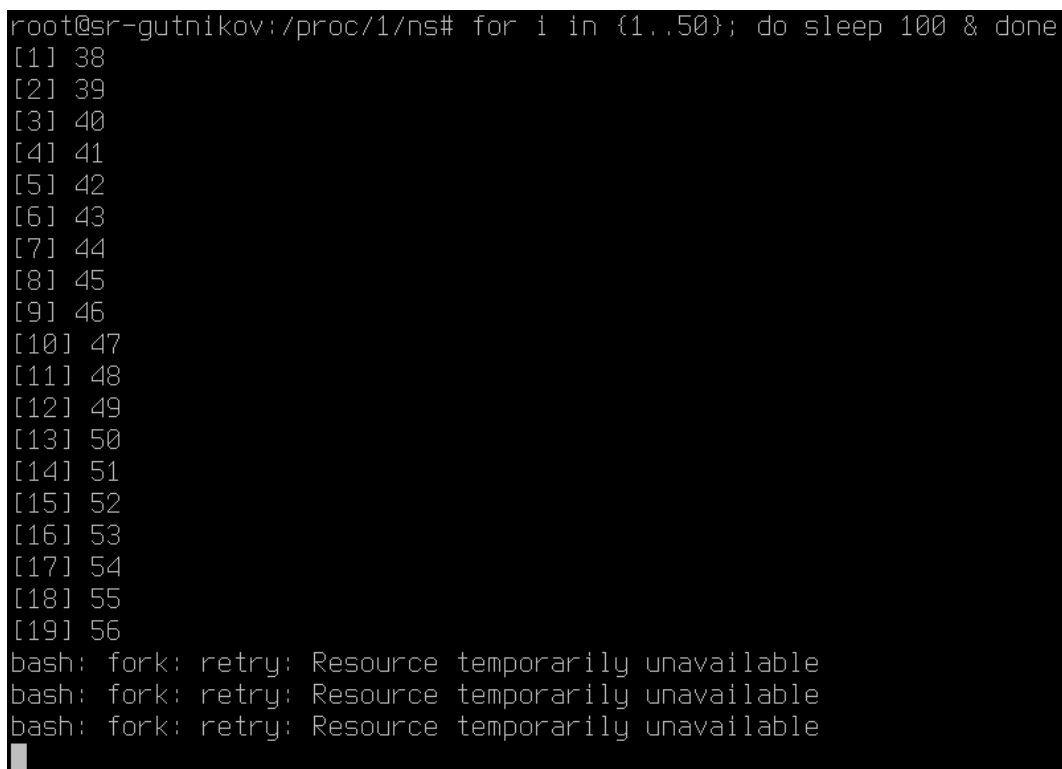


Рисунок 14 – Демонстрация нагрузки процесса.

Как можно увидеть, процесс не нагружает процессор более чем на 10 % процентов, а держится на близких к границе значениях. Максимальное

значение, которое было получено при последовательных запусках команды «ps aux» равнялось 9.9 %.

Дополнительно, мы можем проверить максимально возможное количество запущенных в моменте процессов. Для этого запустим несколько процессов «sleep» одновременно и посмотрим, к какому результату это приведёт (рисунок 15).



```
root@sr-gutnikov:/proc/1/ns# for i in {1..50}; do sleep 100 & done
[1] 38
[2] 39
[3] 40
[4] 41
[5] 42
[6] 43
[7] 44
[8] 45
[9] 46
[10] 47
[11] 48
[12] 49
[13] 50
[14] 51
[15] 52
[16] 53
[17] 54
[18] 55
[19] 56
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
```

Рисунок 15 – Запуск нескольких процессов.

При вызове цикла получилось выполнить 19 вызовов команды sleep, после чего в консоли вывелось сообщение, что ресурсы системы временно недоступны. Нужно понимать, что помимо 19 успешно запущенных процессов есть и родительский процесс командной строки bash, которую мы запускали в самом начале работы – что и даёт суммарное ограничение 20 процессов на контрольную группу.

2.2 OCI bundle и runc

В настоящее время создание контейнеров вручную связано с рисками информационной безопасности, потому что ручная настройка может привести

к ошибкам, которые будут критичны для работы хоста. Данную проблему решает автоматизация.

Перенесём файловую систему в другое место (рисунок 16) и создадим config.json с помощью команды «runc spec».

```
gutnikov@sr-gutnikov:/etc$ cd OCI_bundle/  
gutnikov@sr-gutnikov:/etc/OCI_bundle$ sudo cp -r /tmp/container/rootfs rootfs  
gutnikov@sr-gutnikov:/etc/OCI_bundle$ ls  
rootfs  
gutnikov@sr-gutnikov:/etc/OCI_bundle$
```

Рисунок 16 – Перенос файловой системы.

```
gutnikov@sr-gutnikov:/etc/OCI_bundle$ sudo runc spec  
[sudo] пароль для gutnikov:  
gutnikov@sr-gutnikov:/etc/OCI_bundle$ ls  
config.json rootfs  
gutnikov@sr-gutnikov:/etc/OCI_bundle$
```

Рисунок 17 – Создание config.json.

В данном конфигурационном файле будет следующая информация:

```
{  
  "ociVersion": "1.2.0",  
  "process": {  
    "terminal": true,  
    "user": {  
      "uid": 0,  
      "gid": 0  
    },  
    "args": [  
      "sh"  
    ],  
    "env": [  
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",  
      "TERM=xterm"  
    ],  
    "cwd": "/",  
    "capabilities": {  
      "bounding": [  
        "CAP_AUDIT_WRITE",  
        "CAP_KILL",  
        "CAP_NET_BIND_SERVICE"  
      ],  
      "effective": [  
        "CAP_AUDIT_WRITE",  
        "CAP_KILL",  
        "CAP_NET_BIND_SERVICE"  
      ],  
      "permitted": [  

```

```

        "CAP_AUDIT_WRITE",
        "CAP_KILL",
        "CAP_NET_BIND_SERVICE"
    ]
},
"rlimits": [
    {
        "type": "RLIMIT_NOFILE",
        "hard": 1024,
        "soft": 1024
    }
],
"noNewPrivileges": true
},
"root": {
    "path": "rootfs",
    "readonly": true
},
"hostname": "runc",
"mounts": [
    {
        "destination": "/proc",
        "type": "proc",
        "source": "proc"
    },
    {
        "destination": "/dev",
        "type": "tmpfs",
        "source": "tmpfs",
        "options": [
            "nosuid",
            "strictatime",
            "mode=755",
            "size=65536k"
        ]
    },
    {
        "destination": "/dev/pts",
        "type": "devpts",
        "source": "devpts",
        "options": [
            "nosuid",
            "noexec",
            "newinstance",
            "ptmxmode=0666",
            "mode=0620",
            "gid=5"
        ]
    },
    {
        "destination": "/dev/shm",
        "type": "tmpfs",
        "source": "shm",
        "options": [
            "nosuid",
            "noexec",
            "nodev",
            "mode=1777",
            "size=65536k"
        ]
    }
]

```

```

    ]
  },
  {
    "destination": "/dev/mqueue",
    "type": "mqueue",
    "source": "mqueue",
    "options": [
      "nosuid",
      "noexec",
      "nodev"
    ]
  },
  {
    "destination": "/sys",
    "type": "sysfs",
    "source": "sysfs",
    "options": [
      "nosuid",
      "noexec",
      "nodev",
      "ro"
    ]
  },
  {
    "destination": "/sys/fs/cgroup",
    "type": "cgroup",
    "source": "cgroup",
    "options": [
      "nosuid",
      "noexec",
      "nodev",
      "relatime",
      "ro"
    ]
  }
],
"linux": {
  "resources": {
    "devices": [
      {
        "allow": false,
        "access": "rwm"
      }
    ]
  },
  "namespaces": [
    {
      "type": "pid"
    },
    {
      "type": "network"
    },
    {
      "type": "ipc"
    },
    {
      "type": "uts"
    }
  ]
}

```

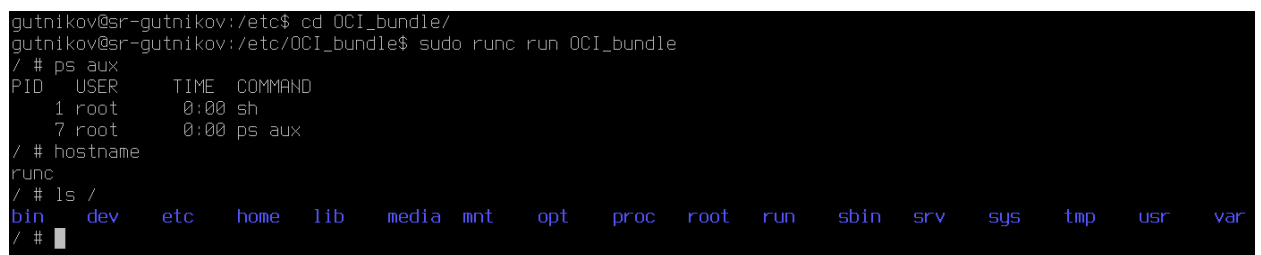
```

        "type": "mount"
    },
    {
        "type": "cgroup"
    }
],
"maskedPaths": [
    "/proc/acpi",
    "/proc/asound",
    "/proc/kcore",
    "/proc/keys",
    "/proc/latency_stats",
    "/proc/timer_list",
    "/proc/timer_stats",
    "/proc/sched_debug",
    "/sys/firmware",
    "/proc/scsi"
],
"readonlyPaths": [
    "/proc/bus",
    "/proc/fs",
    "/proc/irq",
    "/proc/sys",
    "/proc/sysrq-trigger"
]
}

```

Данный конфигурационный файл содержит информацию о контейнере: красным цветом указано, какие пространства имён используются при запуске контейнера, зелёным цветом написано, что будет запущено в рамках контейнера. Ограничения ресурсов прямо не заданы, но есть раздел `capabilities`, в котором есть информация о разрешённых правах (выделено жёлтым).

Запустим контейнер и изучим его работу (рисунок 18). Из вывода команды «`ps aux`» видно, что запустился процесс исполняемого файла `sh`, как и было указано нами выше, имя хоста соответствует имени, указанному в конфигурационном файле, а также важно заметить, что файловая система совпадает с файловой системой, скачанной ранее.



```

gutnikov@sr-gutnikov:/etc$ cd OCI_bundle/
gutnikov@sr-gutnikov:/etc/OCI_bundle$ sudo runc run OCI_bundle
/ # ps aux
PID   USER     TIME   COMMAND
    1  root      0:00   sh
    7  root      0:00   ps aux
/ # hostname
runc
/ # ls /
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
/ #

```

Рисунок 18 – Запуск контейнера.

Если вызвать команду «`runc list`», то выведется набор созданных контейнеров – в нашем случае это будет `OCI_bundle` – а если вызвать команду «`runc state <название контейнера>`», то мы увидим краткую информацию о нашем контейнере в формате `json`.

```
gutnikov@sr-gutnikov:/etc/OCI_bundle$ sudo runc list
[sudo] пароль для gutnikov:
ID          PID          STATUS      BUNDLE      CREATED      OWNER
OCI_bundle  5705         running     /etc/OCI_bundle  2026-04-08T18:37:42.971341823Z  root
gutnikov@sr-gutnikov:/etc/OCI_bundle$ sudo runc state OCI_bundle
{
  "ociVersion": "1.2.0",
  "id": "OCI_bundle",
  "pid": 5705,
  "status": "running",
  "bundle": "/etc/OCI_bundle",
  "rootfs": "/etc/OCI_bundle/rootfs",
  "created": "2026-04-08T18:37:42.971341823Z",
  "owner": ""
}
gutnikov@sr-gutnikov:/etc/OCI_bundle$
```

Рисунок 19 – Изучение контейнера после запуска.

Если сравнивать с первой частью работы, то во второй части процесс создания контейнера формализован и автоматизирован. Если при развёртывании контейнера с помощью технологий ядра нам нужно писать несколько команд для монтирования и ручной конфигурации, то с помощью команды `runc` мы можем запустить автоматическую сборку и запуск контейнера, предварительно описав особые моменты запуска в конфигурационном файле.

2.3 Образы контейнеров

Чтобы создать образ контейнера, напомним соответствующий `Dockerfile`, в котором опишем его структуру (рисунок 20). Первой записью будет запись «FROM», в которой укажем, какую базовую файловую систему мы будем использовать – попробуем выбрать маленькую файловую систему Alpine Linux. Все последующие слои произвольны и могут идти в любом порядке. В нашем случае следом идёт команда `RUN`, выполняющаяся при сборке, а затем идёт команда `CMD`, которая запускается при каждом запуске контейнера.

```
GNU nano 3.2

FROM alpine:3.20

RUN echo "test layer"

CMD ["sh"]
```

Рисунок 20 – Простой Dockerfile.

Соберём образ контейнера и изучим вывод в консоли (рисунок 21). Можно увидеть несколько слоёв, созданных при сборке контейнера: первый слой связан с файловой системой, второй слой вызывает код при сборке образа, а третий – при запуске контейнера.

```
gutnikov@sr-gutnikov:/etc/docker_container$ sudo docker build -t docker-image .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM alpine:3.20
--> cc9071bd1610
Step 2/3 : RUN echo "test layer"
--> Using cache
--> d467c60ab9b0
Step 3/3 : CMD ["sh"]
--> Running in 76f428b93bc4
--> Removed intermediate container 76f428b93bc4
--> fc0e0762c0ae
Successfully built fc0e0762c0ae
Successfully tagged docker-image:latest
gutnikov@sr-gutnikov:/etc/docker_container$
```

Рисунок 21 – Сборка образа.

Вызовем команду `docker` с ключевым словом `history` (рисунок 22). Мы можем посмотреть из каких слоёв состоят, что из себя представляют данные слои и когда они были созданы (рисунок 22).

```
gutnikov@sr-gutnikov:/etc/docker_container$ sudo docker history docker-image
IMAGE          CREATED        CREATED BY          SIZE      COMMENT
fc0e0762c0ae   36 seconds ago /bin/sh -c #(nop)  CMD ["sh"]          0B
d467c60ab9b0   11 minutes ago /bin/sh -c echo "test layer"          0B
cc9071bd1610   2 months ago  CMD ["/bin/sh"]          0B      buildkit.dockerfile.v0
<missing>      2 months ago  ADD alpine-minrootfs-3.20.9-x86_64.tar.gz /...  7.81MB  buildkit.dockerfile.v0
gutnikov@sr-gutnikov:/etc/docker_container$
```

Рисунок 22 – История создания образа `docker-image`.

При вызове команды «docker save» мы сможем создать архив из нашего образа (рисунок 23), чтобы впоследствии изучить его (рисунок 24).

```
gutnikov@sr-gutnikov:/etc/docker_container$ sudo docker save docker-image -o image.tar
gutnikov@sr-gutnikov:/etc/docker_container$ ls
dockerfile  image.tar
gutnikov@sr-gutnikov:/etc/docker_container$
```

Рисунок 23 – Создание архива из образа.

```
gutnikov@sr-gutnikov:/etc/docker_container$ ls
dockerfile  image.tar
gutnikov@sr-gutnikov:/etc/docker_container$ sudo mkdir image_extracted && sudo tar -xf image.tar -C image_extracted
gutnikov@sr-gutnikov:/etc/docker_container$ ls image_extracted/
blobs  index.json  manifest.json  oci-layout  repositories
gutnikov@sr-gutnikov:/etc/docker_container$
```

Рисунок 24 – Извлечение содержимого архива.

Изучим содержимое архива (рисунок 25). Здесь мы видим каталог blobs, в котором лежат ещё архивы (рисунок 26), представляющие из себя слои образа контейнера. Помимо blobs/, в архиве также лежат другие файлы, содержащие информацию об образе (рисунок 27).

```
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ ls
blobs  index.json  manifest.json  oci-layout  repositories
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ echo -e "" && cat index.json && echo -e "\n"
```

Рисунок 25 – Содержимое архива.

```
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ ls blobs/sha256/ -l
итого 7912
-rw-r--r-- 1 root root    1141 апр  9 13:43 0aad4ce40f887115ef70a1ad6f50c4b2f684a25a1faf9925faa9cec80b4de960
-rw-r--r-- 1 root root      401 янв  1 1970 4663daa189a6455206ba6e8883d91229a74551374a99cddf76ce11eb64cff4ca
-rw-r--r-- 1 root root  8089600 апр  9 13:43 f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb
-rw-r--r-- 1 root root    1699 апр  9 13:43 fc0e0762c0ae80a0e63c5d03a51ca0fe2194dbfbd7047136221b5ab6fdcd7bf
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$
```

Рисунок 26 – Архивы слоёв.

```
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ ls
blobs  index.json  manifest.json  oci-layout  repositories
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ echo -e "" && cat index.json && echo -e "\n"
{"schemaVersion":2,"mediaType":"application/vnd.oci.image.index.v1+json","manifests":[{"mediaType":"application/vnd.oci.image.manifest.v1+json","digest":"sha256:4663daa189a6455206ba6e8883d91229a74551374a99cddf76ce11eb64cff4ca","size":401,"annotations":{"io.containerd.image.name":"docker.io/library/docker-image:latest","org.opencontainers.image.ref.name":"latest"}}]}
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ echo -e "" && cat manifest.json && echo -e "\n"
{"Config":"blobs/sha256/fc0e0762c0ae80a0e63c5d03a51ca0fe2194dbfbd7047136221b5ab6fdcd7bf","RepoTags":["docker-image:latest"],"Layers":["blobs/sha256/f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb"],"LayerSources":[{"sha256":"f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb","mediaType":"application/vnd.oci.image.layer.v1.tar","size":8089600,"digest":"sha256:f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb"}]}
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ echo -e "" && cat oci-layout && echo -e "\n"
{"imageLayoutVersion":"1.0.0"}
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ echo -e "" && cat repositories && echo -e "\n"
{"docker-image":{"latest":{"f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb"}}}
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$
```

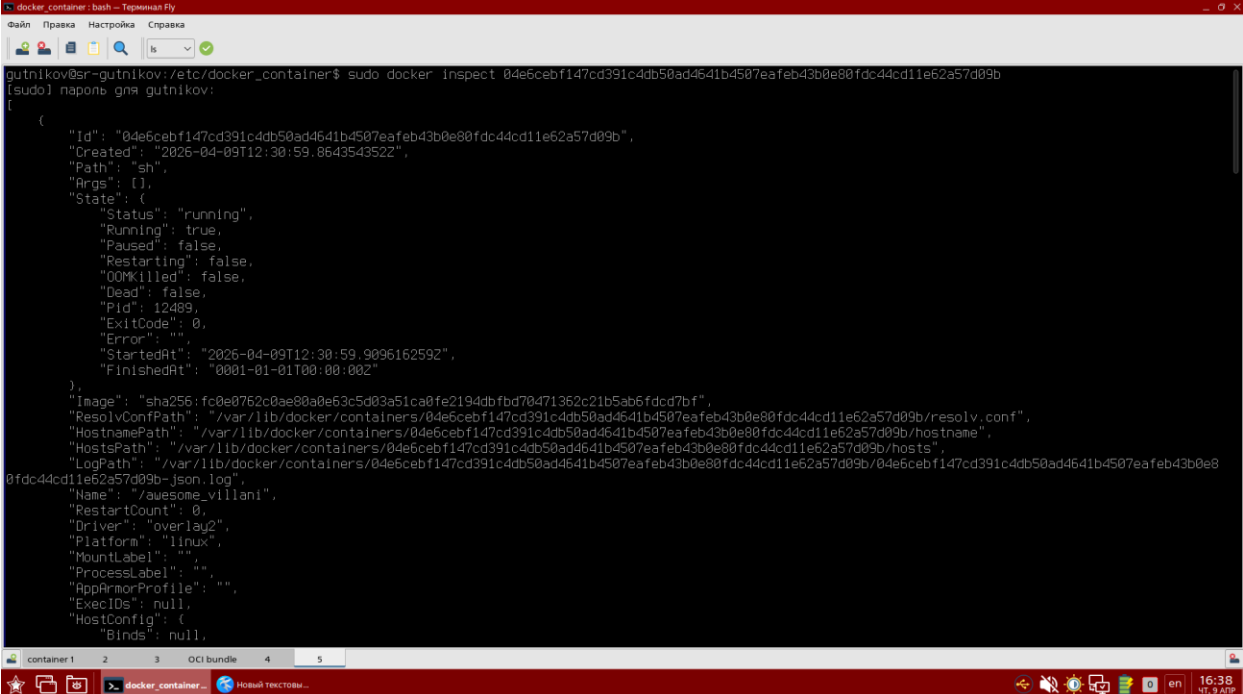
Рисунок 27 – Содержимое файлов.

Так, `oci-layout` показывает версию образа, что важно для демонстрации совместимости. Файл `index.json` показывает точку входа в контейнер – в нашем случае это второй архив, показанный на рисунке 26. В `manifest.json` используется для обратной совместимости. В `repositories` находится информация об используемых репозиториях.

Запустим контейнер (рисунок 28) и изучим его с помощью `inspect`. Посмотрим вывод (рисунок 29). После запуска контейнера мы получили длинный идентификатор, по которому можно идентифицировать контейнер, что мы и будем делать.

```
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
docker-image  latest    fc0e0762c0ae   2 hours ago    7.81MB
<none>        <none>    bc461a5e3f16   2 hours ago    7.81MB
<none>        <none>    796321eca530   2 hours ago    117MB
<none>        <none>    93133336412b   2 hours ago    117MB
debian        12        3f011f36e5ef   3 days ago     117MB
alpine        3.20      cc9071bd1610   2 months ago   7.81MB
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$ sudo docker run -dit docker-image sh
04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b
gutnikov@sr-gutnikov:/etc/docker_container/image_extracted$
```

Рисунок 28 – Запуск контейнера.



```
gutnikov@sr-gutnikov:/etc/docker_container$ sudo docker inspect 04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b
[sudo] пароль гня gutnikov:
[
  {
    "Id": "04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b",
    "Created": "2026-04-09T12:30:59.864354352Z",
    "Path": "sh",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 12489,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-09T12:30:59.909616259Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:fc0e0762c0ae80a0e63c5d03a51ca0fe2194dbfbd70471362c21b5ab6f6cd7bf",
    "ResolvConfPath": "/var/lib/docker/containers/04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/hostname",
    "HostsPath": "/var/lib/docker/containers/04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/hosts",
    "LogPath": "/var/lib/docker/containers/04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/04e6ceb147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b-json.log",
    "Name": "/awesome_villani",
    "RestartCount": 0,
    "Driver": "overlay2",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,

```

Рисунок 29 – Изучение контейнера.

В выводе данной команды можно найти информацию о контейнере.
Выделим цветом нужную нам информацию.

```
[
  {
    "Id": "04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b",
    "Created": "2026-04-09T12:30:59.864354352Z",
    "Path": "sh",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 12489,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-09T12:30:59.909616259Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:fc0e0762c0ae80a0e63c5d03a51ca0fe2194dbfbd70471362c21b5ab6fcdcd7bf",
    "ResolvConfPath":
"/var/lib/docker/containers/04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/resolv.c
onf",
    "HostnamePath":
"/var/lib/docker/containers/04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/hostname",
    "HostsPath":
"/var/lib/docker/containers/04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/hosts",
    "LogPath":
"/var/lib/docker/containers/04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b/04e6ce
bf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b-json.log",
    "Name": "/awesome_villani",
    "RestartCount": 0,
    "Driver": "overlay2",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
      "ContainerIDFile": "",
      "LogConfig": {
        "Type": "json-file",
        "Config": {}
      },
      "NetworkMode": "bridge",
      "PortBindings": {},
      "RestartPolicy": {
        "Name": "no",
        "MaximumRetryCount": 0
      },
      "AutoRemove": false,
      "VolumeDriver": "",
      "VolumesFrom": null,
```

```

"ConsoleSize": [
  37,
  156
],
"CapAdd": null,
"CapDrop": null,
"CgroupnsMode": "private",
"Dns": [],
"DnsOptions": [],
"DnsSearch": [],
"ExtraHosts": null,
"GroupAdd": null,
"IpcMode": "private",
"Cgroup": "",
"Links": null,
"OomScoreAdj": 0,
"PidMode": "",
"Privileged": false,
"PublishAllPorts": false,
"ReadOnlyRootfs": false,
"SecurityOpt": null,
"UTSMode": "",
"UsersMode": "",
"ShmSize": 67108864,
"Runtime": "runc",
"Isolation": "",
"CpuShares": 0,
"Memory": 0,
"NanoCpus": 0,
"CgroupParent": "",
"BlkioWeight": 0,
"BlkioWeightDevice": [],
"BlkioDeviceReadBps": [],
"BlkioDeviceWriteBps": [],
"BlkioDeviceReadIOps": [],
"BlkioDeviceWriteIOps": [],
"CpuPeriod": 0,
"CpuQuota": 0,
"CpuRealtimePeriod": 0,
"CpuRealtimeRuntime": 0,
"CpusetCpus": "",
"CpusetMems": "",
"Devices": [],
"DeviceCgroupRules": null,
"DeviceRequests": null,
"MemoryReservation": 0,
"MemorySwap": 0,
"MemorySwappiness": null,
"OomKillDisable": null,
"PidsLimit": null,
"Ulimits": [],
"CpuCount": 0,
"CpuPercent": 0,
"IOMaximumIOps": 0,
"IOMaximumBandwidth": 0,
"MaskedPaths": [
  "/proc/asound",
  "/proc/acpi",
  "/proc/interrupts",

```

```

        "/proc/kcore",
        "/proc/keys",
        "/proc/latency_stats",
        "/proc/timer_list",
        "/proc/timer_stats",
        "/proc/sched_debug",
        "/proc/scsi",
        "/sys/firmware",
        "/sys/devices/virtual/powercap"
    ],
    "ReadonlyPaths": [
        "/proc/bus",
        "/proc/fs",
        "/proc/irq",
        "/proc/sys",
        "/proc/sysrq-trigger"
    ]
},
"GraphDriver": {
    "Data": {
        "ID": "04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b",
        "LowerDir":
"/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696-
init/diff:/var/lib/docker/overlay2/5d27e7d707348271bc30e080f126a3e904e909a0d4c981cd1f34a53288410a69/di
ff",
        "MergedDir":
"/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696/merged",
        "UpperDir":
"/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696/diff",
        "WorkDir":
"/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696/work"
    },
    "Name": "overlay2"
},
"Mounts": [],
"Config": {
    "Hostname": "04e6cebf147c",
    "Domainname": "",
    "User": "",
    "AttachStdin": false,
    "AttachStdout": false,
    "AttachStderr": false,
    "Tty": true,
    "OpenStdin": true,
    "StdinOnce": false,
    "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
    ],
    "Cmd": [
        "sh"
    ],
    "Image": "docker-image",
    "Volumes": null,
    "WorkingDir": "/",
    "Entrypoint": null,
    "OnBuild": null,
    "Labels": {}
},
"NetworkSettings": {

```

```

    "Bridge": "",
    "SandboxID": "9cf5fcf7d8b8579755806b57677b003ab145d234ac9d5a7e70ee42fa853dc482",
    "SandboxKey": "/var/run/docker/netns/9cf5fcf7d8b8",
    "Ports": {},
    "HairpinMode": false,
    "LinkLocalIPv6Address": "",
    "LinkLocalIPv6PrefixLen": 0,
    "SecondaryIPAddresses": null,
    "SecondaryIPv6Addresses": null,
    "EndpointID": "da7f272767940d362c1b7c49bd53335567dbbb7ecc784655b6ca89e047702d51",
    "Gateway": "172.17.0.1",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "IPAddress": "172.17.0.3",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "MacAddress": "1e:e9:c7:90:0f:71",
    "Networks": {
      "bridge": {
        "IPAMConfig": null,
        "Links": null,
        "Aliases": null,
        "MacAddress": "1e:e9:c7:90:0f:71",
        "DriverOpts": null,
        "GwPriority": 0,
        "NetworkID": "aa50c0eebf4e0a3538868bd34c73ca3ebdc34583da24c0fbbbc547f69b989e7",
        "EndpointID":
"da7f272767940d362c1b7c49bd53335567dbbb7ecc784655b6ca89e047702d51",
        "Gateway": "172.17.0.1",
        "IPAddress": "172.17.0.3",
        "IPPrefixLen": 16,
        "IPv6Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "DNSNames": null
      }
    }
  }
}
]

```

Здесь мы можем получить информацию о слоях Overlay2. Это важно, потому что в этой файловой системе изменения файлов происходят в более высоких слоях, причём файлы в нижестоящих слоях остаются без изменений.

Создадим файл в контейнере (рисунок 30) и покажем, что он появится в UpperLayer (рисунок 31).

```

gutnikov@sr-gutnikov:/var/lib$ sudo docker exec -it 04e6cebf147cd391c4db50ad4641b4507eafeb43b0e80fdc44cd11e62a57d09b sh
[sudo] пароль для gutnikov:
/ # ls
bin    dev    etc    home  lib    media mnt    opt    proc  root  run    sbin  srv    sys    tmp    usr    var
/ # cd tmp
/tmp # echo "temporary file to be found" > tmp_file.txt
/tmp # ls
tmp_file.txt
/tmp # █

```

Рисунок 30 – Создание файла в контейнере.

```

root@sr-gutnikov:/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696# ls
diff link lower merged work
root@sr-gutnikov:/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696# ls diff
root tmp
root@sr-gutnikov:/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696# ls diff/tmp/
tmp_file.txt
root@sr-gutnikov:/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696# cat diff/tmp/tmp_file.txt
temporary file to be found
root@sr-gutnikov:/var/lib/docker/overlay2/8f801421c75f20af120b894f1ffb81733f96ff2b0a671dd3d838bf2966dac696# █

```

Рисунок 31 – Нахождение файла на хосте вне контейнера.

Теперь создадим другой Dockerfile, в котором создадим инструкции для копирования файла с секретом и его удаления (рисунок 32). Секрет будет выглядеть следующим образом, как на рисунке 33.

```

GNU nano 3.2

FROM alpine:3.20

COPY secret /secret.txt

RUN rm /secret.txt

CMD ["sh"]
█

```

Рисунок 32 – Второй Dockerfile.

```

root@sr-gutnikov:/etc/docker_container2# ls
dockerfile secret
root@sr-gutnikov:/etc/docker_container2# cat secret
SOME_SORT_OF_A_SECRET_123456
root@sr-gutnikov:/etc/docker_container2# █

```

Рисунок 33 – Демонстрация «секрета».

Соберём новый образ (рисунок 34) и изучим его. По выводу консоли видно, что образ стал состоять из 4 слоёв:

1. Базовая файловая система;
2. Копирование секрета;
3. Удаление секрета;
4. Запуск изолированного процесса.

```

gutnikov@sr-gutnikov:/etc/docker_container2$ sudo docker build -t docker-image2 .
[sudo] пароль для gutnikov:
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/4 : FROM alpine:3.20
--> cc9071bd1610
Step 2/4 : COPY secret /secret.txt
--> 8ef11d4a8780
Step 3/4 : RUN rm /secret.txt
--> Running in 8dd78cee4e1c
--> Removed intermediate container 8dd78cee4e1c
--> db72277e9b3a
Step 4/4 : CMD ["sh"]
--> Running in 4709500c2e78
--> Removed intermediate container 4709500c2e78
--> 9b975a1e75b7
Successfully built 9b975a1e75b7
Successfully tagged docker-image2:latest
gutnikov@sr-gutnikov:/etc/docker_container2$

```

Рисунок 34 – Сборка нового образа.

С помощью `docker history` мы сможем изучить созданный ранее образ и посмотреть слои (рисунок 35). Так мы можем обнаружить сам факт создания секрета.

```

root@sr-gutnikov:/var/lib/docker/overlay2/04878d6e8f1a4aeb1e38ef8c94f98a4ce81a7f5e0a5071c0ece28412a20b26cb# docker history docker-image2
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
9b975a1e75b7   About an hour ago /bin/sh -c #(nop)  CMD ["sh"]      0B
db72277e9b3a   About an hour ago /bin/sh -c rm /secret.txt      0B
8ef11d4a8780   About an hour ago /bin/sh -c #(nop) COPY file:0d5c1c639902107b... 29B
cc9071bd1610   2 months ago    CMD ["/bin/sh"]      0B      buildkit.dockerfile.v0
<missing>      2 months ago    ADD alpine-minirootfs-3.20.9-x86_64.tar.gz /... 7.81MB  buildkit.dockerfile.v0
root@sr-gutnikov:/var/lib/docker/overlay2/04878d6e8f1a4aeb1e38ef8c94f98a4ce81a7f5e0a5071c0ece28412a20b26cb#

```

Рисунок 35 – Исследование нового образа.

Сохраним новый образ в виде архива и извлечём его содержимое (рисунок 36).

```

root@sr-gutnikov:/etc/docker_container2# tar -xf image2.tar -C image2_extracted/
root@sr-gutnikov:/etc/docker_container2# cd image2_extracted/
root@sr-gutnikov:/etc/docker_container2/image2_extracted# ls
blobs index.json manifest.json oci-layout repositories
root@sr-gutnikov:/etc/docker_container2/image2_extracted#

```

Рисунок 36 – Извлечение архива.

Теперь есть возможность исследовать архив на наличие секретов в различных слоях. Разархивируем все архивы в `blobs/` (рисунок 37) и попытаемся найти в одном из них обнаруженный ранее `secret.txt` (рисунок 38).

```

root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256# for f in *; do mkdir "dir_$(echo $f | tr -d _)" && tar -xvf $f -C "dir_$(echo $f | tr -d _)"; done
tar: Это не похоже на tar-архив
tar: Пропускается до следующего заголовка
tar: Завершение работы с состоянием неисправности из-за возникших ошибок
tar: Это не похоже на tar-архив
tar: Завершение работы с состоянием неисправности из-за возникших ошибок
tar: Это не похоже на tar-архив
tar: Завершение работы с состоянием неисправности из-за возникших ошибок
tar: Это не похоже на tar-архив
tar: Пропускается до следующего заголовка
tar: Завершение работы с состоянием неисправности из-за возникших ошибок
tar: Это не похоже на tar-архив
tar: Пропускается до следующего заголовка
tar: Завершение работы с состоянием неисправности из-за возникших ошибок
root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256# ls
3d2c93261eb3efff331effff650964448d5c1120b74f3d1bf9c8476e5eeee92f  dir_42fc4b53795afabc2525305cf9f0914273c44b8700824bd8c70ca32c12d9efcd
42fc4b53795afabc2525305cf9f0914273c44b8700824bd8c70ca32c12d9efcd  dir_673dfcbb11fbec5a06a478f72c6a6c20d2c8df1e683c64fc9aa6a65d0f2981f3
673dfcbb11fbec5a06a478f72c6a6c20d2c8df1e683c64fc9aa6a65d0f2981f3  dir_6ac6fec148c35af288d2f5204452c9884bfb84e2d403f4bf7e519293d01616a5
6ac6fec148c35af288d2f5204452c9884bfb84e2d403f4bf7e519293d01616a5  dir_8cd28f8c56bf03d6f85f32657057dc3663734ffa7e70c3137ddf7c4d0c62b01
8cd28f8c56bf03d6f85f32657057dc3663734ffa7e70c3137ddf7c4d0c62b01  dir_9b975a1e75b7d2ae8abe0c87325c05e7d0318e9a0a5f7b413b07cf566aea364c
9b975a1e75b7d2ae8abe0c87325c05e7d0318e9a0a5f7b413b07cf566aea364c  dir_d223ccd6cf0dad0b76ecf01b272b68b2377d408d2545b9e1201e2b00bf64f6e
d223ccd6cf0dad0b76ecf01b272b68b2377d408d2545b9e1201e2b00bf64f6e  dir_f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb
dir_3d2c93261eb3efff331effff650964448d5c1120b74f3d1bf9c8476e5eeee92f  f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb
root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256#

```

Рисунок 37 – Разархивирование.

```

root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256# ls
3d2c93261eb3efff331effff650964448d5c1120b74f3d1bf9c8476e5eeee92f  dir_42fc4b53795afabc2525305cf9f0914273c44b8700824bd8c70ca32c12d9efcd
42fc4b53795afabc2525305cf9f0914273c44b8700824bd8c70ca32c12d9efcd  dir_673dfcbb11fbec5a06a478f72c6a6c20d2c8df1e683c64fc9aa6a65d0f2981f3
673dfcbb11fbec5a06a478f72c6a6c20d2c8df1e683c64fc9aa6a65d0f2981f3  dir_6ac6fec148c35af288d2f5204452c9884bfb84e2d403f4bf7e519293d01616a5
6ac6fec148c35af288d2f5204452c9884bfb84e2d403f4bf7e519293d01616a5  dir_8cd28f8c56bf03d6f85f32657057dc3663734ffa7e70c3137ddf7c4d0c62b01
8cd28f8c56bf03d6f85f32657057dc3663734ffa7e70c3137ddf7c4d0c62b01  dir_9b975a1e75b7d2ae8abe0c87325c05e7d0318e9a0a5f7b413b07cf566aea364c
9b975a1e75b7d2ae8abe0c87325c05e7d0318e9a0a5f7b413b07cf566aea364c  dir_d223ccd6cf0dad0b76ecf01b272b68b2377d408d2545b9e1201e2b00bf64f6e
d223ccd6cf0dad0b76ecf01b272b68b2377d408d2545b9e1201e2b00bf64f6e  dir_f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb
dir_3d2c93261eb3efff331effff650964448d5c1120b74f3d1bf9c8476e5eeee92f  f5b2c0456bf1080bb3f0617295df9721aa4aa27a536bba8e301c3af8d530c9cb
root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256# ls dir_3d2c93261eb3efff331effff650964448d5c1120b74f3d1bf9c8476e5eeee92f/
secret.txt
root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256# cat dir_3d2c93261eb3efff331effff650964448d5c1120b74f3d1bf9c8476e5eeee92f/secret.txt
SOME SORT OF A SECRET 123456
root@sr-gutnikov:/etc/docker_container2/image2_extracted/blobs/sha256#

```

Рисунок 38 – Нахождение секрета.

Несмотря на удаление «секрета», мы всё равно смогли его обнаружить, так как каждый последующий слой в образе не влияет на содержимое предыдущего слоя. Именно поэтому при выполнении команды «`RUN rm /secret.txt`» файл остаётся неизменным в слое, в котором он был создан.

3. Дополнительное задание

Выполним дополнительное задание и запустим контейнеры в особом и обычном режимах, чтобы сравнить, чем будут отличаться процессы.

Запустим контейнер с флагом `--privileged` и посмотрим на его особенности. Перед этим сформируем новый Dockerfile, в котором поменяем файловую систему на большую (рисунок 39). Добавим особый слой, в котором поставим пакеты `libcap2-bin`, чтобы в контейнере была возможность использовать инструмент `capsh`.


```
GNU nano 3.2

FROM debian:12

RUN apt-get update && apt-get install -y libcap2-bin

CMD ["sh"]
```

Рисунок 39 – Новый Dockerfile для тестирования режимов.

Соберём образ нашего контейнера и посмотрим на вывод консоли (рисунок 40). Вывод оказался большим и полностью не поместился на экране.

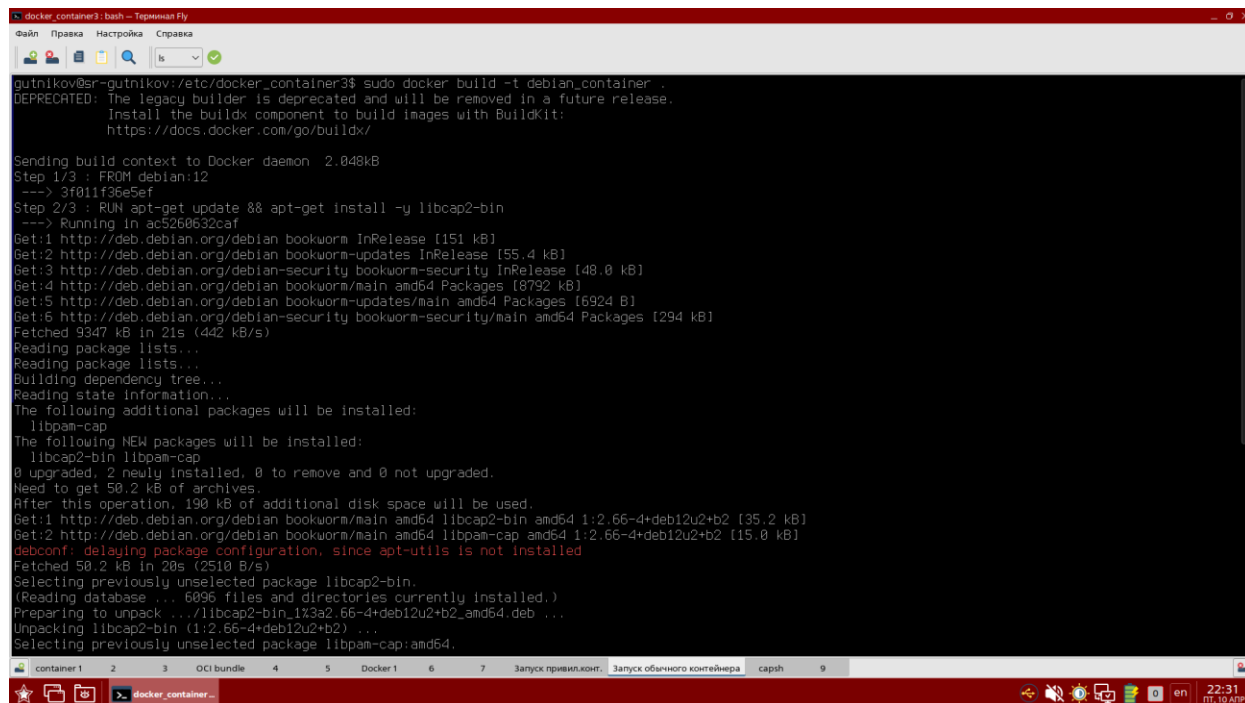


Рисунок 40 – Сборка нового образа.

Командой «capsh --print» мы получим информацию о правах различных процессов. Так, например, посмотрим права внутри контейнеров и сравним их (рисунок 41 и 42).

```

gutnikov@sr-gutnikov:/$ sudo docker exec -it b89988b3ddf8b582212c950c7badd08a4eb7dc683d68305c34c88069340862a bash
[sudo] naponov gna gutnikov:
root@b89988b3ddf8:/# capsh --print
Current: cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_mknod,cap_audit_write,cap_setfcap=ep
Bounding set =cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_raw,cap_sys_chroot,cap_mknod,cap_audit_write,cap_setfcap
Ambient set =
Current IAB: !cap_dac_read_search,!cap_linux_immutable,!cap_net_broadcast,!cap_net_admin,!cap_ipc_lock,!cap_ipc_owner,!cap_sys_module,!cap_sys_rawio,!cap_sys_ptrace,!cap_sys_pacct,!cap_sys_admin,!cap_sys_boot,!cap_sys_nice,!cap_sys_resource,!cap_sys_time,!cap_sys_tty_config,!cap_lease,!cap_audit_control,!cap_mac_override,!cap_mac_admin,!cap_syslog,!cap_wake_alarm,!cap_block_suspend,!cap_audit_read,!cap_perfmom,!cap_bpf,!cap_checkpoint_restore
Securebits: 00/0x0/1'b0 (no-new-privs=0)
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
secure-no-ambient-raise: no (unlocked)
uid=0(root) euid=0(root)
gid=0(root)
groups=0(root)
Gauged mode: HYBRID (4)
root@b89988b3ddf8:/#

```

Рисунок 41 – Права непривилегированного контейнера.

```

root@a3edeb33a618:/# capsh --print
Current: =ep
Bounding set =cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_fsetid,cap_kill,cap_setgid,cap_setuid,cap_setpcap,cap_linux_immutable,cap_net_bind_service,cap_net_broadcast,cap_net_admin,cap_net_raw,cap_ipc_lock,cap_ipc_owner,cap_sys_module,cap_sys_rawio,cap_sys_chroot,cap_sys_ptrace,cap_sys_pacct,cap_sys_admin,cap_sys_boot,cap_sys_nice,cap_sys_resource,cap_sys_time,cap_sys_tty_config,cap_mknod,cap_lease,cap_audit_write,cap_audit_control,cap_setfcap,cap_mac_override,cap_mac_admin,cap_syslog,cap_wake_alarm,cap_block_suspend,cap_audit_read,cap_perfmom,cap_bpf,cap_checkpoint_restore
Ambient set =
Current IAB:
Securebits: 00/0x0/1'b0 (no-new-privs=0)
secure-noroot: no (unlocked)
secure-no-suid-fixup: no (unlocked)
secure-keep-caps: no (unlocked)
secure-no-ambient-raise: no (unlocked)
uid=0(root) euid=0(root)
gid=0(root)
groups=0(root)
Gauged mode: HYBRID (4)
root@a3edeb33a618:/#

```

Рисунок 42 – Права привилегированного контейнера.

Если сравнивать вывод команды `capsh`, то видно, что у привилегированного контейнера большее число возможных разрешений (большой `bounding set`), но самое важное, что в нём присутствует разрешение `cap_sys_admin`, которое позволяет монтировать файловые системы, управлять пространствами имён, настраивать сетевые интерфейсы и прочие привилегированные операции – данного `sarability` нет в непривилегированном контейнере, причём для него стоит чёткий запрет для наследования в «Current IAB».

Проблема эскалации привилегий в контейнерах связана с неправильной настройкой и выдачей широкого спектра прав. Попробуем получить доступ из привилегированного контейнера с помощью монтирования блочных устройств в виртуальную файловую систему.

Чтобы сделать это, обнаружим блочные устройства по пути `/dev` – здесь будут располагаться все устройства, обнаруженные в системе. Будем искать устройства с помощью утилиты «`lsblk`», которая выведет список устройств (рисунок 43). Эти устройства видны в `/dev` (рисунок 44).

```
gutnikov@sr-gutnikov:/etc/docker_container3$ sudo docker exec -it a3edeb33a618ef0c0d23237d9e9f3ae78f2ea0fd3f8b9937b42db2c2fd4d85a04 bash
root@a3edeb33a618:/# lsblk -f
NAME        FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sda
|-sda1
|          12.2G    43% /etc/hosts
|          /etc/hostname
|          /etc/resolv.conf
|-sda2
|-sda5
|          [SWAP]
sr0
root@a3edeb33a618:/#
```

Рисунок 43 – Вывод всех блочных устройств.

```
root@a3edeb33a618:/# ls /dev | grep sda
sda
sda1
sda2
sda5
root@a3edeb33a618:/#
```

Рисунок 44 – Обнаружение блочных устройств в /dev.

Примонтируем первый раздел диска (sda1) к файловой системе контейнера (рисунок 45) и создадим файл на базовой файловой системе. Убедимся в том, что мы действительно смогли получить доступ к внешней файловой системе (рисунок 46).

```
root@a3edeb33a618:/# mkdir /tmp/host && mount /dev/sda1 /tmp/host && cd /tmp/host && ls
bin  dev  home  initrd.img.old  lib32  libx32  lost+found  mnt  parsec  proc  run  srv  tmp  var  vmlinuz.old
boot  etc  initrd.img  lib  lib64  logwatch.7.12-3_all.deb  media  opt  parsecfs  root  sbin  sys  usr  vmlinuz
root@a3edeb33a618:/tmp/host# cd /home/gutnikov/ && touch file
root@a3edeb33a618:/# ls
Desktop      '$'\320\222\320\270\320\264\320\265\320\276'
Desktops    '$'\320\224\320\276\320\272\321\203\320\274\320\265\320\275\321\202\321\213'
SystemWallpapers '$'\320\227\320\260\320\263\321\200\321\203\320\267\320\272\320\270'
file        '$'\320\230\320\267\320\276\320\261\321\200\320\260\320\266\320\265\320\275\320\270\321\217'
git         '$'\320\234\321\203\320\267\321\213\320\272\320\260'
new_file    '$'\320\236\320\261\321\211\320\265\320\264\320\276\321\201\321\202\321\203\320\277\320\275\321\213\320\265'
new_file2   '$'\320\250\320\260\320\261\320\273\320\276\320\275\321\213'
root@a3edeb33a618:/tmp/host/home/gutnikov# echo "This file was created in the Debian container"
This file was created in the Debian container
root@a3edeb33a618:/tmp/host/home/gutnikov# echo "This file was created in the Debian container" > file
root@a3edeb33a618:/tmp/host/home/gutnikov#
```

Рисунок 45 – Монтирование файловой системы в контейнере.

```
gutnikov@sr-gutnikov:/etc/docker_container3$ cd /home/gutnikov/ && ls
Desktop Desktops file git new_file new_file2 SystemWallpapers Вуего Документы Загрузки Изображения Музыка Общедоступные Шаблоны
gutnikov@sr-gutnikov:~$ cat file
This file was created in the Debian container
gutnikov@sr-gutnikov:~$
```

Рисунок 46 – Нахождение файла на основной файловой системе.

Чтобы избежать эскалации привилегий, нужно:

- Не использовать флаг `--privileged` и чётко определять, какие capabilities будут нужны и не приведут к уязвимости контейнера;

- Использовать `seccomp` для ограничения набора доступных системных вызовов и не отключать эту опцию самостоятельно;
- Запуск контейнеров от лица обычного пользователя с помощью флага `-u`.

Покажем, что из себя представляет контейнер с опцией «`--security-opt seccomp=unconfined`». Эта опция отключает защиту от системных вызовов, что подвергает риску безопасность контейнера. Запустим контейнер с этим флагом (рисунок 47).

```
gutnikov@sr-gutnikov:~$ sudo docker run -dit --security-opt seccomp=unconfined debian_container bash
45232ae3c213ca4ec553f9c2bd19e2bb467f77007e66fa72fd8518ab9fb21629
gutnikov@sr-gutnikov:~$
```

Рисунок 47 – Запуск контейнера с небезопасным параметром.

Попробуем вызвать в данном контейнере функцию `unshare` (рисунок 48). Как можно увидеть, вызов был успешно обработан, что позволило создать процесс в изолированной среде (в пространстве имён пользователей). Если же мы попробуем вызвать эту команду в обычном контейнере, который мы запускали ранее, то заданные правила не позволят это сделать (рисунок 49).

```
gutnikov@sr-gutnikov:~$ sudo docker exec -it 45232ae3c213ca4ec553f9c2bd19e2bb467f77007e66fa72fd8518ab9fb21629 bash
root@45232ae3c213:/# unshare --user
$ ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
$ hostname
45232ae3c213
$
```

Рисунок 48 – Вызов `unshare` в небезопасном контейнере.

```
gutnikov@sr-gutnikov:~$ sudo docker exec -it b88988b3ddf8b5882212c950c7badd08a4eb7dc683d58305c34c88069340862a bash
[sudo] пароль для gutnikov:
root@b88988b3ddf8:/# unshare --user
unshare: unshare failed: Operation not permitted
root@b88988b3ddf8:/# hostname
b88988b3ddf8
root@b88988b3ddf8:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@b88988b3ddf8:/#
```

Рисунок 49 – Вызов `unshare` в безопасном контейнере.

4. Вывод

В результате выполнения данной практической работы было изучено, что из себя представляет контейнер в Linux, как его запускать и чем отличаются способы запуска между собой. Были получены знания в области безопасности контейнеров, механизмов их защиты.